If you define a default policy but then also call `RequireCors()` or add an `[EnableCors]` attribute, then both policies are applied. This can get confusing, so I recommend not applying a default CORS policy in the middleware and specifying the policy at the route group or endpoint level. Alternatively, if you do want to apply a policy to your whole app, avoid applying individual policies to endpoints as well.

Whether you choose to use a single default CORS policy or multiple policies, you need to configure the CORS policies for your application in the call to `AddCors`. Many options are available when configuring CORS. In the next section I provide an overview of the possibilities.

29.3.4 Configuring CORS policies

Browsers implement the cross-origin policy for security reasons, so you should carefully consider the implications of relaxing any of the restrictions they impose. Even if you enable cross-origin requests, you can still control what data cross-origin requests can send and what your API returns. For example, you can configure

- The origins that may make a cross-origin request to your API
- The HTTP verbs (such as `GET`, `POST`, and `DELETE`) that can be used
- The headers the browser can send
- The headers the browser can read from your app's response
- Whether the browser will send authentication credentials with the request